ConnectFourAI

Assignment 2 Report

CP468-D - Artificial Intelligence

Wilfrid Laurier University

View Interactive GUI

Group 7

Farhan Chowdhury

Manahil Bashir

Morad Mohammad

Noah Samarita

Safdar Shukur

1. Introduction

Connect Four serves as a classic game environment for exploring adversarial search spaces, perfect information constraints, and deterministic transition mechanics. The game features a combination of deep branching factors and gravity constraints that require a balanced integration of systematic algorithms and heuristic valuations.

This project explores these principles by creating a centralized game engine and implementing three intelligent agents of increasing complexity: a baseline random agent, a rule-based agent, and an advanced zero-sum tree-search explorer.

2. System Design

2.1 State Space and Board Representation

The board environment is modeled as a factored grid matrix of dimensions 6 rows by 7 columns. Row coordinates map strictly from 0 up to 5, where row index 0 represents the absolute bottom cell of the grid layout and row 5 captures the top height ceiling. Cells are represented explicitly as integers: 0 denotes an empty cell, 1 represents Player 1 tokens, and 2 identifies Player 2 discs.

2.2 Core Engine

The engine operates deterministically through primary operations:

- **legal_moves():** Scans the top index (row 5) of all 7 columns. It returns a dynamic array of available column indices where a piece can safely fall.
- **apply_move(col):** Simulates downward physics gravity by scanning a target column from row index 0 up to 5. It locks the piece into the first available empty cell, alternates the player turn identifier, updates the move counter, and handles illegal out-of-bounds selections by raising explicit ValueError constraints.
- **winner():** Executes bidirectional orientation matrix checks scanning the board for 4 consecutive identical non-zero discs across all axes (Horizontal, Vertical, Main-Diagonal, and Anti-Diagonal). If a win is confirmed, it returns the winning player ID (1 or 2). If the grid is full with no winner, it returns 0 (Draw); otherwise, it evaluates to None to indicate an ongoing game.

- **clone():** Generates a complete state deep copy of the board array, move counts, and active players. This utility allows tree-search agents to simulate hypothetical futures without polluting the actual match state.

3. AI Agents

3.1 Agent 1: Baseline Uniform Random Agent

The random agent establishes a lower performance boundary. It queries the engine's valid column list and picks a move using an independent, isolated random object initialized via an optional integer seed parameter.

3.2 Agent 2: Logic-Driven Rule-Based Agent

Agent 2 implements a multi-tiered conditional priority hierarchy designed to evaluate immediate state utilities without incurring exponential branch exploration costs:

1. **Immediate Wins:** Loops through actions using deep copies. If any column yields an immediate victory via winner(), that column is picked instantly.
2. **Opponent Blocks:** If no win is present, it simulates the opponent's next turn. It identifies any column where the opponent could immediately connect a line of four and executes a defensive block.
3. **Spatial Center Preference:** Lacking immediate win or block constraints, it scores columns by their distance to the central column (Column 3) to maximize long-term vector alignment possibilities.
4. **Contiguous Threat Scoring & Tie-Breaking:** Remaining moves are filtered by calculating the maximum length of contiguous matching pieces intersecting that position. Surviving ties are resolved cleanly using an isolated random seed instance.

3.3 Agent 3: Adversarial Alpha-Beta Minimax Agent

Agent 3 executes an adversarial tree-search optimization algorithm using a zero-sum framing. Minimax evaluates alternating plies where MAX aims to maximize the evaluation function, while MIN moves to minimize it. To manage computation costs, the

agent sets a standard search horizon limit of depth d=4.

To prune redundant subtrees, the search updates bounds tracking alpha (MAX's lower bound) and beta (MIN's upper bound). Branches are cut immediately whenever beta is less than or equal to alpha. Definite terminal wins evaluate to true mathematical infinity, while losses evaluate to negative infinity. For non-terminal leaf nodes sitting at the depth limit, a sliding matrix heuristic counts line configurations based on optimized positive priority multipliers for non-terminal structures, and opponent clusters are subtracted as penalties.

4. Experimental Results & Evaluation

System testing was automated over batches of 30 games per agent pairing using independent initialized parameter distributions. Turn priority was alternated evenly (~15 matches starting as Player 1 vs Player 2). Compute speed was evaluated in wall-clock time using high-precision counters. Every agent was constructed with its own seed for each individual game so the results are exactly reproducible.

Test machine: Intel Core i7-13620H, 3.8 GB RAM, Ubuntu 22.04.5 LTS (Linux 6.8.0, x86_64), Python 3.10.12.

Matchup Pairing (P1 vs. P2)	P1 Win %	P2 Win %	Draw %	Avg. Decision Speed
Random vs. Rule-Based	3.33%	96.67%	0.00%	0.2555 ms
Rule-Based vs. Minimax (D4)	10.00%	86.67%	3.33%	33.4204 ms
Minimax (D4) vs. Random	100.00%	0.00%	0.00%	42.5260 ms

Our metrics validate a clear hierarchy of strategic capability: Random Agent < Rule-Based Agent < Minimax Search Agent.

5. Discussion

5.1 Performance Hierarchy

- **Validated Strength Ordering:** Results follow the expected path: **Random (weakest) < Rule-Based (middle) < Minimax (strongest)**.
- **Rule-Based vs. Random (29–1):** Proves that immediate win/block checks and center preference provide a massive step up over random play. The single random win shows the baseline isn't completely toothless.
- **Minimax Dominance:** Swept Random 30–0, and dominated Rule-Based 26–3 (with 1 draw).

5.2 First-Mover Advantage & Forcing Sequences

- **The 9-Move Deterministic Signature:** When Minimax played first against Rule-Based, *every single game* (all 15) ended in a Minimax win in exactly 9 moves. This consistent signature shows depth-4 minimax is strong enough to execute a flawless forcing line against predictable heuristics.
- **Random Unpredictability:** Against Random, Minimax's first-mover wins varied (7 to 17 moves) because erratic play prevents a fixed forcing sequence.
- **First-Move Asymmetry:** All of Rule-Based's points (3 wins, 1 draw) against Minimax occurred *only* when Rule-Based moved first. It never won or drew from the second-mover position.

5.3 Decision Times & Pruning Mechanics

- **Lookahead Overhead:** Random and Rule-Based agents operate in a fraction of a millisecond, effectively achieving O(1) lookahead. In contrast, the Minimax agent requires an average of **33 to 43 ms per move** to expand its depth-4 game tree.
- **Pruning Efficiency:** Minimax was notably slower against the Random agent (42.53 ms) than against the Rule-Based agent (33.42 ms). This is because the Random agent's chaotic move selection creates un-ordered, high-branching game trees, whereas the Rule-Based agent's structured play steers the game into predictable positions, allowing **Alpha-Beta pruning** to trigger cutoffs significantly earlier in the search process.

6. Team Member Contributions

- Safdar Md Abdus Shukur: Requirement 1 (Game Engine), Requirement 2 — Agent 1 (Random Agent) and Web GUI
- Morad Mohammad: Requirement 2 — Agent 2 (Rule-Based Agent)
- Noah Samarita: Requirement 2 — Agent 3 (Minimax Agent)
- Farhan Chowdhury: Requirement 3 (Experimental Evaluation) and the Report (Requirement 4)
- Manahil Bashir: Requirement 3 (Experimental Evaluation), done together with Farhan on his system, and Requirement 5 (Demonstration Video)

6. References & Acknowledgements

- Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020.
- Python Software Foundation. *The Python Language Reference & Standard Library Documentation*. https://www.python.org
- ECMA International. *ECMAScript® Language Specification*. https://tc39.es/ecma262/
- Tailwind CSS. *Tailwind Engine Core CDN Architecture*. https://tailwindcss.com
- Lucide Open Source Community. *The Lucide Vector Asset Library*. https://lucide
- Music: Pixabay. Mykola Sosin .https://pixabay.com/users/mfcc-28627740/